SMART COLLEGE MANAGEMENT PLATFORM

Project report submitted
in partial fulfillment of the requirement for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

MALEPATI PREM KUMAR REDDY (23UECS1122)
CHERUKULA LOKESHWAR REDDY (23UECS1039)
CH.VAMSI KRISHNA REDDY (23UECS0127)

10211CS212 - WEB AND MOBILE APPLICATION DEVELOPMENT

SUMMER 2026-2027

Under the guidance of
Dr. Manikandan N K, M.E., Ph.D
Assistant Professor - Senior



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

October, 2026

CERTIFICATE

It is certified that the work contained in the project report titled "Smart College Management Platform" by MALEPATI PREM KUMAR REDDY (23UECS1122), CHERUKULA LOKESHWAR REDDY (23UECS1039), CH.VAMSI KRISHNA REDDY (23UECS0127) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
Dr. Manikandan N K, M.E., Ph.D
Assistant Professor - Senior
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr.Sagunthala R&D
Institute of Science & Technology
October, 2026

Signature of Head/Assistant Head
of the Department

Dr. M. Kavitha / Dr. T. Kujani
Professor & Head / Associate Professor & Assistant
Head
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
October, 2026

Signature of the Dean

Dr. S. P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
October, 2026

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

MALEPATI PREM KUMAR REDDY

Date: / /

(Signature)

CHERUKULA LOKESHWAR REDDY

Date: / /

(Signature)

CH.VAMSI KRISHNA REDDY

Date: / /

APPROVAL SHEET

This project report entitled “Smart College Management Platform” by MALEPATI PREM KUMAR REDDY (23UECS1122), CHERUKULA LOKESHWAR REDDY (23UECS1039), CH.VAMSI KRISHNA REDDY (23UECS0127) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners **Handling faculty**

Dr. Manikandan N K, M.E., Ph.D

Date: / /

Place:

ABSTRACT

Smart College Management Platform, implemented as *veltech-ams-mini*, is a unified student web application for academic attendance and campus engagement. It combines the attendance functions of Unieve-Attandance-AI with the events and community functions of Unieven-AI. A responsive workspace presents student identity, current and upcoming classes, subject attendance, registered courses, results and earned credits alongside campus events, posts, reels, connections and faculty-created forms.

The system uses a React/Vite interface, a Node.js/Express campus service and a private Python/-FastAPI academic service. Its academic adapter reads supported authenticated AMS pages while preserving report headings, semester divisions and source timestamps. University records remain separate from editable social profiles. Results include a cumulative CGPA line graph, labelled published or calculated values, and semester-level PDF and Excel exports. Persistent campus sessions, incremental synchronization and bounded background work support continuity while limiting unnecessary requests.

The supplied project record reports automated regression, browser and Docker integration checks, together with earlier authorized read-only observations of AMS profile, course, attendance, timetable, results and credit structures for one account. Missing sections are identified explicitly. The application provides a unified campus workspace with a clear boundary between university-owned academic records and campus-managed activities.

Keywords: Academic Management System, attendance, campus events, student community, source-aware synchronization, progressive web application.

LIST OF FIGURES

3.1 Architecture and sitemap of the project 4
3.2 Data Flow Diagram 5
3.3 Home page of the project 6
3.4 Campus login page 7
3.5 Form validation in the project 8
3.6 Academic records presented in the results dashboard 9
3.7 Node.js campus webserver implementation 10
3.8 Three-tier application and private academic integration 11
3.9 Reactive event-registration form 12
3.10 React routing to subject attendance 13
3.11 Campus and academic services in Docker Compose 14
3.12 Recorded Docker build and local SMTP verification 15

5.1 Local web application preview 17

8.1 Selected campus-service security source 20

9.1 Mobile event discovery 21
9.2 Mobile community feed 21

LIST OF TABLES

2.1 Software specification 3

4.1 Representative verification outcomes 16

6.1 Fictional credit-weighted example: 8.40 CGPA gives 79% 18

LIST OF ACRONYMS AND ABBREVIATIONS

AMS	Academic Management System
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CSRF	Cross-Site Request Forgery
DOM	Document Object Model
GPA	Grade Point Average
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
OTP	One-Time Password
PDF	Portable Document Format
PWA	Progressive Web Application
QR	Quick Response code
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface
XLSX	Office Open XML spreadsheet format

TABLE OF CONTENTS

Page.No

ABSTRACT **iv**

LIST OF FIGURES **v**

LIST OF TABLES **vi**

LIST OF ACRONYMS AND ABBREVIATIONS **vii**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the project 1

1.3 Project Domain 1

1.4 Scope of the Project 2

1.5 Methodology 2

2 REQUIREMENT SPECIFICATION **3**

2.1 User characteristics 3

2.2 Dependencies 3

2.3 Hardware specification 3

2.4 Software specification 3

3 WEBSITE DESIGN **4**

3.1 Sitemap 4

3.2 Design Phase 5

3.2.1 Data Flow Diagram 5

3.3 Front End and Back End Design 6

3.3.1 Home Page 6

3.3.2 Campus Login and Session Restoration 7

3.3.3 Form Validation 8

3.3.4 Parsing AMS Webpages Using Python and the DOM 9

3.3.5 Creation of Webserver Using Node.js 10

3.3.6 Design of the Three-Tier Application with MongoDB 11

3.3.7 Design of Reactive Forms in React 12
3.3.8 Routing and Navigation in React 13
3.3.9 Creation of Microservices 14
3.3.10 Deployment of Microservices 15

4 TESTING **16**
4.1 Testing 16
4.1.1 Test Result 16
4.1.2 Test Bugs 16

5 WEBSITE LAUNCH **17**

6 RESULTS AND DISCUSSIONS **18**
6.1 Website performance 18
6.2 Security 18
6.3 Responsiveness and mobile-friendliness 18

7 CONCLUSION AND FUTURE ENHANCEMENTS **19**
7.1 Conclusion 19
7.2 Future Enhancements 19

8 SOURCE CODE **20**

9 SCREENSHOTS **21**

References **22**

Chapter 1

INTRODUCTION

1.1 Introduction

Smart College Management Platform, implemented as *veltech-ams-mini*, combines the Unieven-AI event/community project and the Unieve-Attandance-AI attendance project into one student web application. It is an independent student project for Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, Chennai [1]. The combined application presents supported university AMS records [2], daily classes, campus activities and student interaction through a common responsive interface.

Students can review attendance and academic progress without leaving the campus workspace. Events, posts, reels, connections and faculty forms complement the academic functions. Official college identity remains separate from editable social profiles. Source timestamps and unavailable states help distinguish current, retained and missing information.

1.2 Aim of the project

The aim is to provide a clear, unified student workspace with the following objectives:

1. Present supported AMS student details, class timing, attendance, results and credits.
2. Combine event registration and community participation with scoped staff management.
3. Improve session continuity, synchronization feedback and phone navigation.
4. Provide repeatable Windows/Docker setup and permitted PDF/Excel exports.

1.3 Project Domain

The project belongs to educational information systems [3], campus event management and student community applications. React/Vite provides the interface, Node.js/Express manages campus workflows, and a private Python/FastAPI service reads supported AMS reports. MongoDB stores campus records and academic snapshots.

1.4 Scope of the Project

The academic scope includes source student facts, subject attendance and calendars, Day/Week/Full timetable, registered courses, results, assessments and earned credits. Fees, receipts, documents and examinations depend on supported source records. Campus functions include free/paid event workflows, tickets, posts/reels, comments, saves, connections, faculty forms, notifications and settings.

Official course submission remains an AMS portal handoff. Paid tickets require configured, server-verified payment confirmation. Missing placement or university data remains developing/unavailable. No supported independent Home of VelTech private API is configured; the retired parent portal is unused.

1.5 Methodology

Development consolidates the two repositories, separates campus and academic services, maps inspected source tables, and builds reusable responsive views. Validation combines automated tests, fictional API/browser workflows and earlier authorized read-only AMS observations. Successful sections appear incrementally; failed sections retain accepted records with their original timestamps. Bounded work queues and visibility-aware polling reduce redundant requests.

Chapter 2

REQUIREMENT SPECIFICATION

2.1 User characteristics

Students view their own academic records and join campus activities. Faculty create scoped forms; organizers manage assigned club or department events. Administrators manage permitted roles, notices, calendar policies and app feedback. Social-profile edits do not grant staff permissions.

2.2 Dependencies

Academic retrieval requires reachable AMS pages and a valid university session. Atlas needs database permissions and network access [4]. Email, payments, camera, location and notifications require their configured providers or browser permissions.

2.3 Hardware specification

Development uses a Windows computer supporting Docker Desktop and Node.js. Hosting requires persistent storage, network access and HTTPS. Students use a current phone or desktop browser; server capacity requires deployment-specific testing.

2.4 Software specification

Technology	**Project use**
HTML, CSS, JavaScript	Page structure, styling and interaction.
React, Vite	Component interface and PWA build [5, 6].
Tailwind, Bootstrap, React Router	Responsive layout and navigation.
Axios, Node.js, Express	HTTP requests and campus REST API.
Python, FastAPI, HTTPX	Private academic integration [7].
APScheduler, PyMongo, Mongoose	Background tasks and MongoDB access.
JWT, HttpOnly cookies	Private authentication and browser sessions.
Docker Compose, PowerShell	Isolated services and Windows setup [8].

Table 2.1: Software specification

Chapter 3

WEBSITE DESIGN

3.1 Sitemap

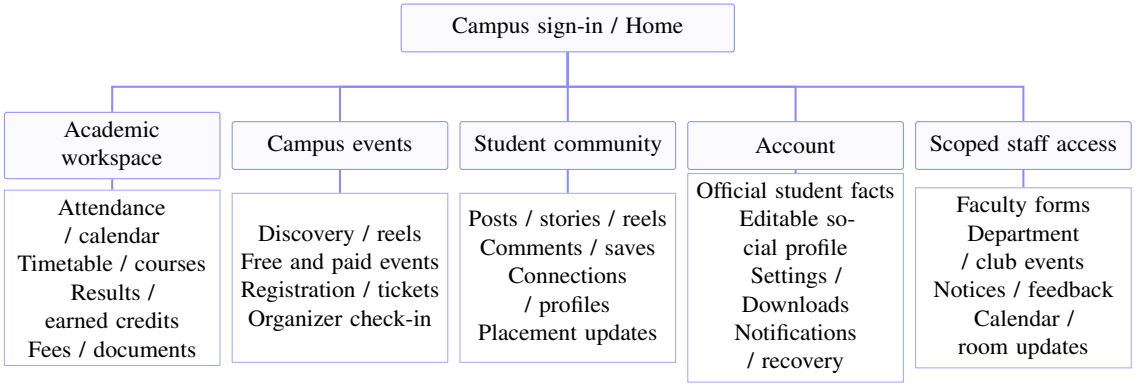


Figure 3.1: Architecture and sitemap of the project

Figure 3.1 shows the navigation structure of *Smart College Management Platform*. The shared Home workspace leads to academic records, campus events, student community and account settings. Attendance, timetable, registered courses, results, earned credits, fees and documents belong to the academic area. Events lead to registration and private tickets; community links posts, reels and connections. Faculty, organizer and administrator tools appear only within their assigned permissions. Official student details remain separate from editable social profiles. The screen captures in this chapter use the implemented project with fictional demonstration records.

3.2 Design Phase

3.2.1 Data Flow Diagram

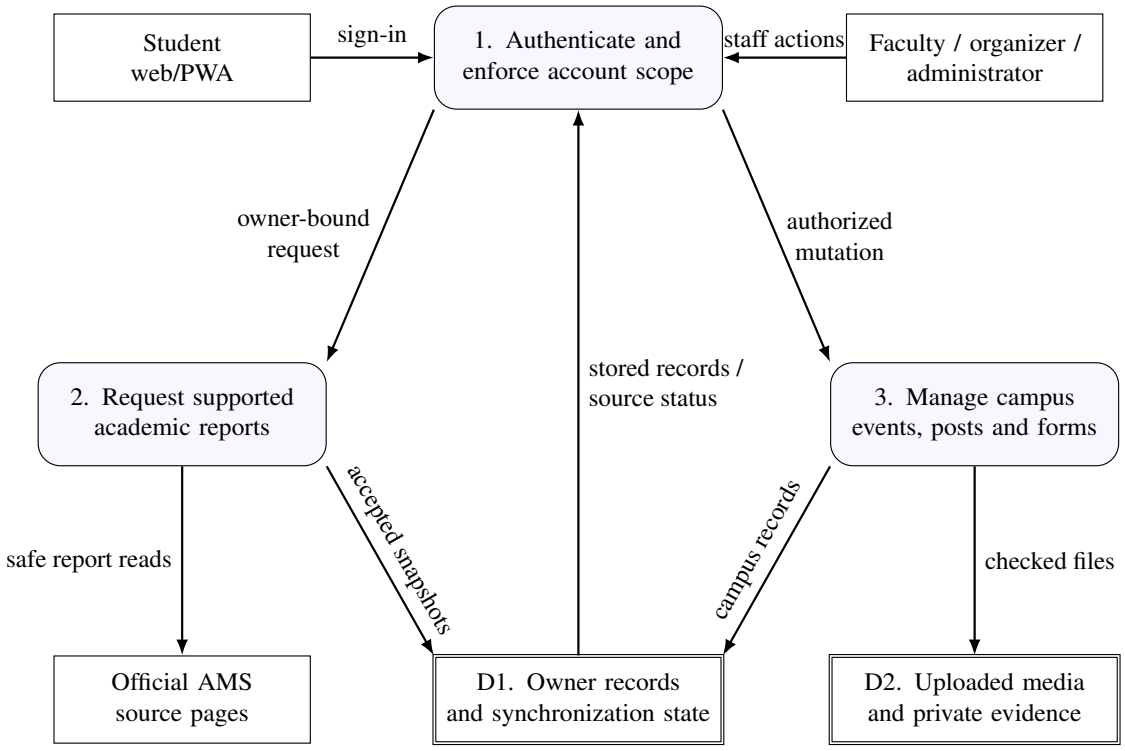


Figure 3.2: Data Flow Diagram

Figure 3.2 shows the flow of academic and campus information. Authentication establishes account ownership before protected requests are accepted. The academic adapter reads supported AMS reports and retains accepted snapshots with source timestamps. Campus services manage events, social interactions and faculty forms under role and scope checks. Uploaded media and private evidence have separate visibility rules. The browser receives the records and their synchronization status; a failed source section retains its last accepted data rather than creating missing university records.

3.3 Front End and Back End Design

Front End

React interface: academic dashboard, events and community.

Back End

Node.js campus API, Python academic service and MongoDB.

3.3.1 Home Page

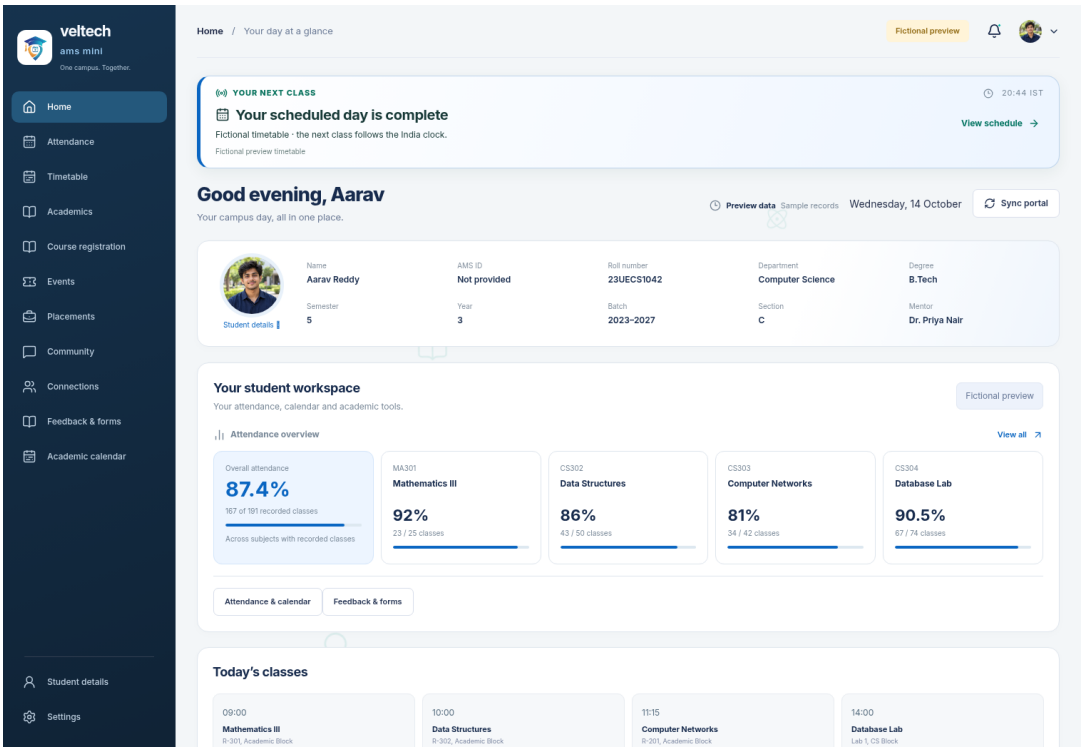


Figure 3.3: Home page of the project

Figure 3.3 shows the student Home workspace. The current or next class uses India time and the available source timetable. A compact identity panel presents available student facts and a portrait. Overall and subject attendance, academic shortcuts and notices follow. Present or absent colours use supplied attendance marks; missing information remains distinct. The demonstration screenshot illustrates the interface and does not represent an author's university record.

3.3.2 Campus Login and Session Restoration

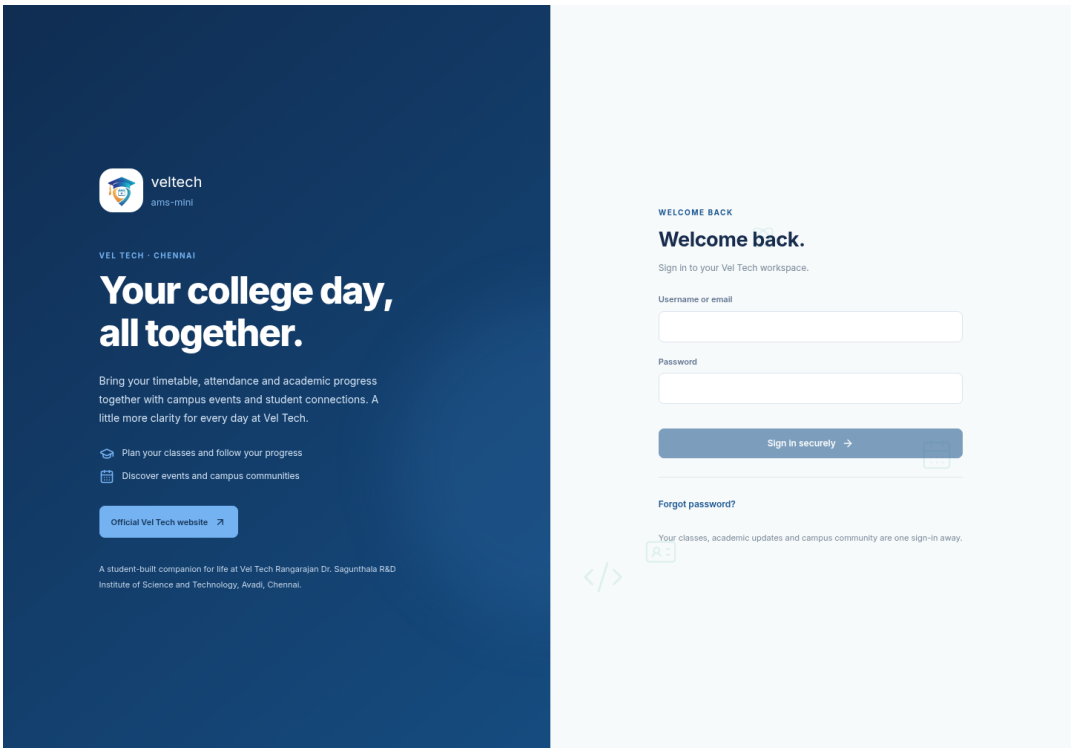


Figure 3.4: Campus login page

Figure 3.4 shows the common sign-in page. Students, faculty and administrators enter their username or email and password in the same interface; persisted account permissions determine the destination. Staff credentials remain separate from a student's AMS connection. The client restores a valid campus session before showing login and offers retry for temporary connection failures. University verification appears only when required by AMS. The forgot-password flow uses a verified address and a browser-bound, single-use code; it does not change the university password.

3.3.3 Form Validation

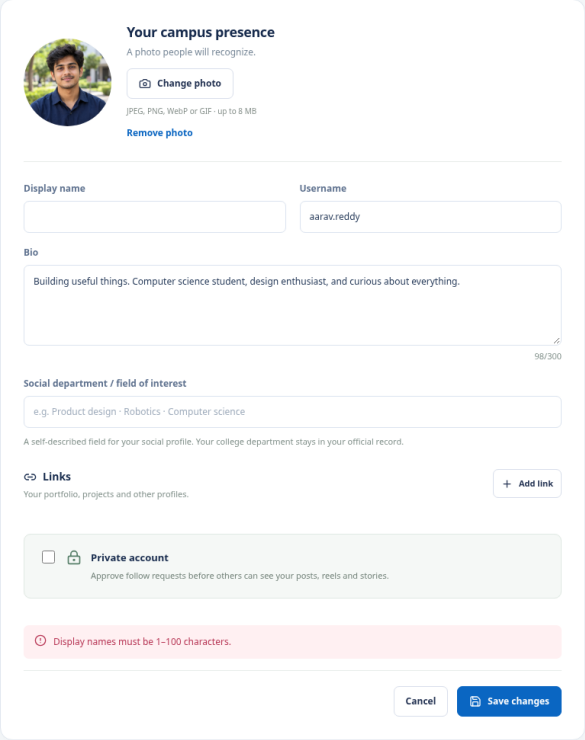


Figure 3.5: Form validation in the project

Figure 3.5 shows an actual validation response from the React social-profile form. A whitespace-only display name is rejected after trimming, and the form keeps the student on the same screen to correct it. Username format, field lengths and social links are also checked. Server-side ownership and permission checks protect saved changes. The editable social identity is separate from official college records, so changing a display name, bio or social department cannot alter AMS identity or grant a staff role.

3.3.4 Parsing AMS Webpages Using Python and the DOM

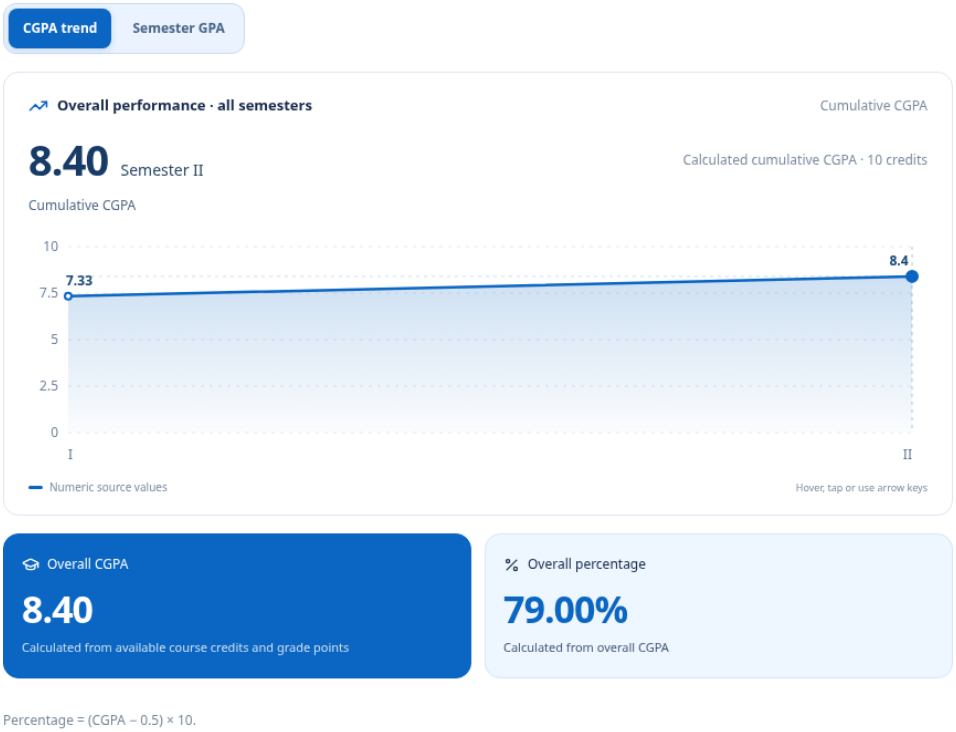


Figure 3.6: Academic records presented in the results dashboard

Figure 3.6 illustrates the academic presentation layer after supported records are normalized. The Python adapter uses verified-TLS requests and structured HTML parsing to read labelled AMS reports. Semester, report and examination-session divisions are preserved. Published summaries take precedence over calculated estimates; eligible course credits and grade points support the cumulative graph. The fictional example displays 8.40 CGPA and 79% using the requested conversion. Unsupported page layouts are marked unavailable, while accepted records retain their original timestamp. Official university submissions are not performed by this parser.

3.3.5 Creation of Webserver Using Node.js

```
if(process.argv[1] && path.resolve(process.argv[1])===fileURLToPath(import.meta.url)) {
  dotenv.config({path:process.env.CAMPUS_ENV_FILE || path.resolve(path.dirname(
      fileURLToPath(import.meta.url)),'.env')});
  const mongoUri=process.env.MONGODB_URI;if(!mongoUri) throw new Error('Set MONGODB_URI
      in campus/.env');
  await mongoose.connect(mongoUri);
  // Enforce uniqueness before accepting registrations or account links.
  await Promise.all([User.init(),Event.init(),Registration.init(),Session.init(),
      AcademicLink.init(),IdentityChallenge.init(),IdentityProof.init(),CampusMedia.init
      (),Follow.init(),AcademicCalendarEntry.init(),TimetableRoomOverride.init(),
      AppFeedback.init(),initializeNotificationModels()]);
  // Other initialization and scheduled jobs omitted from this excerpt.
  const app=createCampusApp({mail});app.listen(Number(process.env.CAMPUS_PORT || 4100),
      process.env.CAMPUS_HOST || '127.0.0.1',()=>console.log('veltech-ams-mini API
      started'));
}
```

Figure 3.7: Node.js campus webserver implementation

Figure 3.7 shows an excerpt from the project's Express server. The service connects to MongoDB, initializes required models and starts the campus application. Its REST routes provide sessions, academic bridging, events, posts, forms, notifications and management. Protected requests enforce ownership, role scope and CSRF checks; request and file limits protect relevant inputs. Private responses use no-store handling. The built React interface can be served by the same campus container, keeping the browser and campus API on a common public origin.

3.3.6 Design of the Three-Tier Application with MongoDB

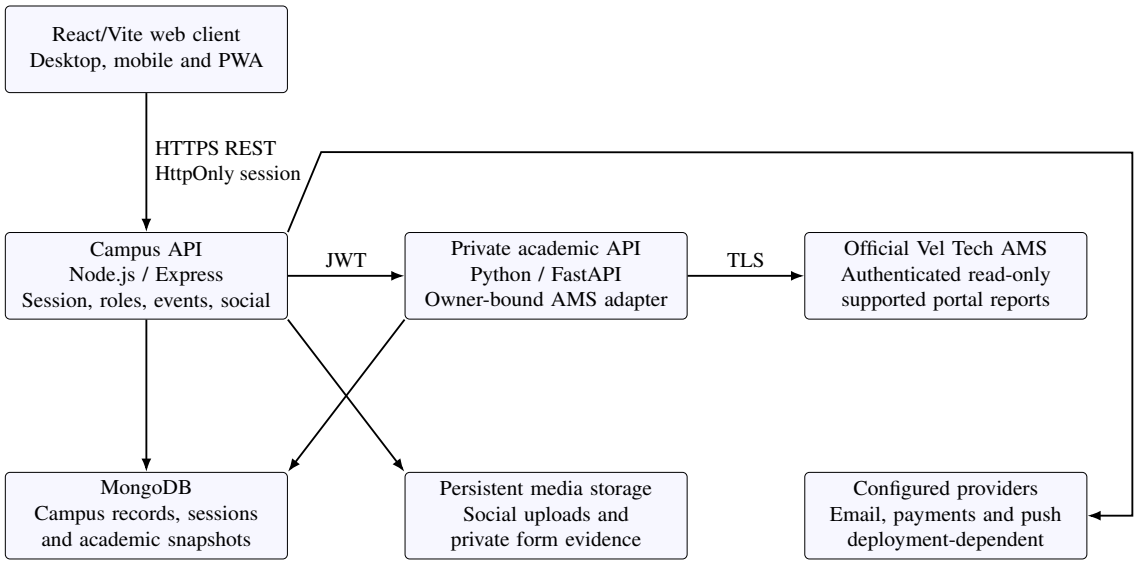


Figure 3.8: Three-tier application and private academic integration

Figure 3.8 shows the project's presentation, application and persistence responsibilities [9]. React forms the presentation tier. Node.js handles campus workflows and bridges owner-bound academic requests to the private FastAPI service. MongoDB stores campus records, sessions and academic snapshots, while persistent media storage holds checked uploads. Official academic data is excluded from public social queries. The AMS connection is a supported authenticated read path; this diagram describes the student project and does not claim to represent the university's internal server architecture.

3.3.7 Design of Reactive Forms in React

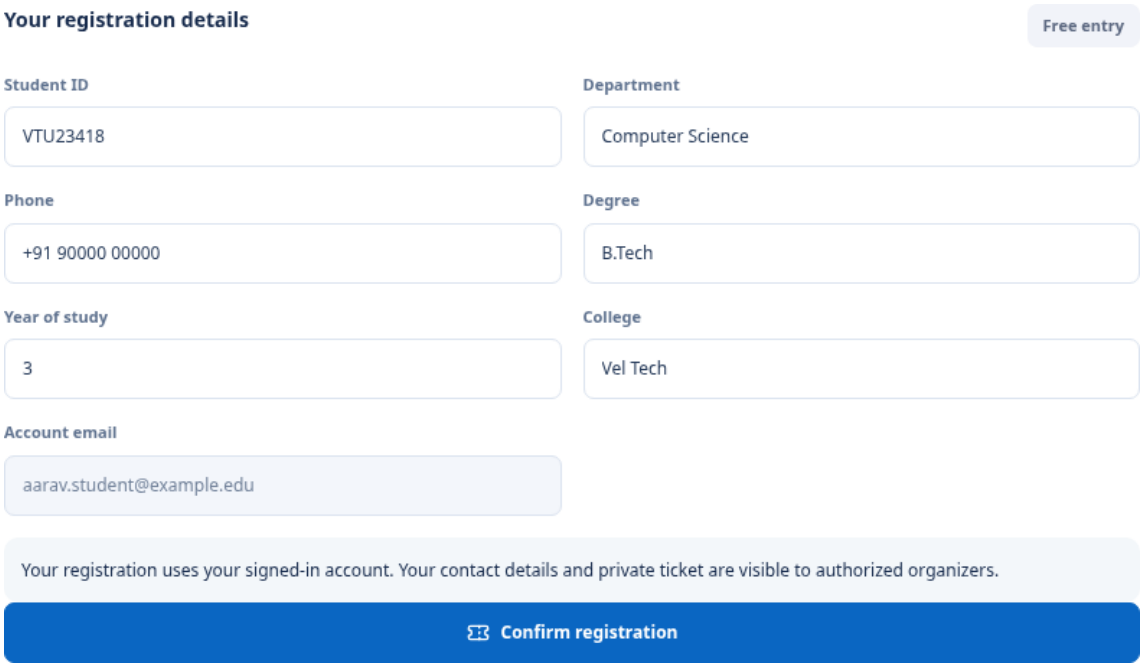


Figure 3.9: Reactive event-registration form

Figure 3.9 shows the controlled event-registration form. Student ID, department, phone, degree, study year and college are collected for the selected event; the account email comes from the signed-in workspace. Input changes update React state and the submit action exposes progress, confirmation or a recoverable error. Free events proceed to registration. Paid events require a configured provider and server-verified payment before issuing a ticket. The displayed event and student values are fictional; the report screenshot does not establish a real payment or event booking.

3.3.8 Routing and Navigation in React

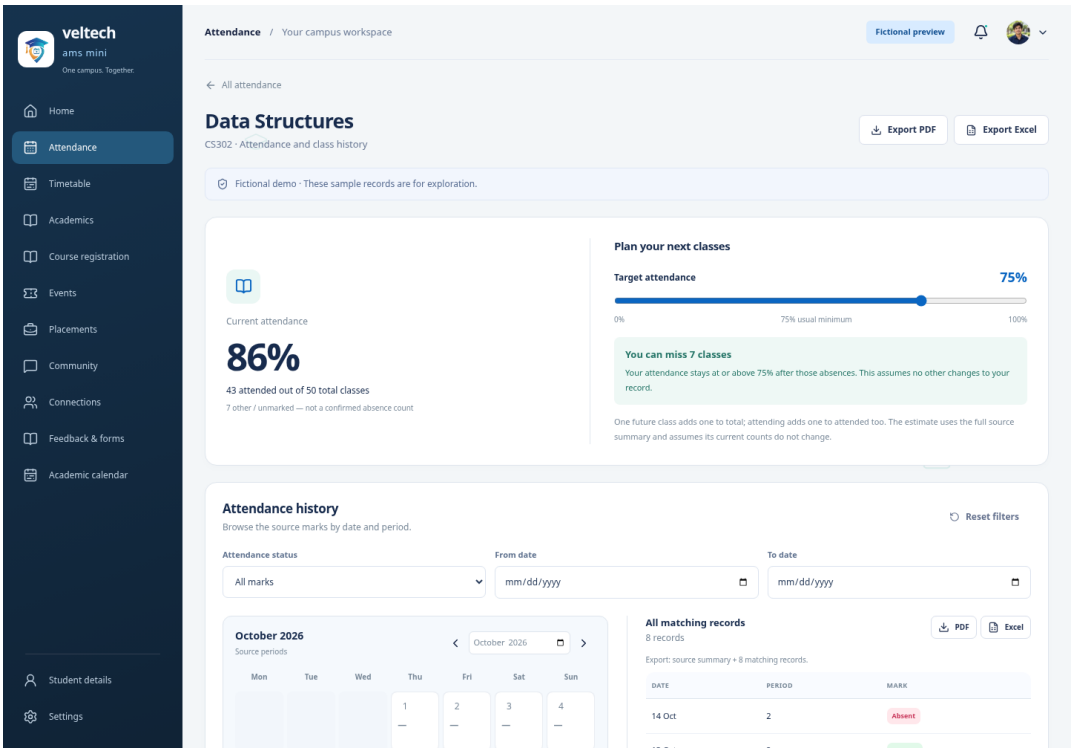


Figure 3.10: React routing to subject attendance

Figure 3.10 shows a dedicated subject-attendance route. Selecting a subject opens its full history, planning information and dated records while preserving the common navigation shell. The calendar and filters narrow the displayed history without changing source totals. Desktop navigation becomes a bottom bar with More on phones. Events, tickets, profiles and forms also use dedicated routes; comments remain associated with the active post or reel. These navigation choices let students move between academic and campus work without losing the relevant context.

3.3.9 Creation of Microservices

```
services:
  academic:
    build:
      dockerfile: docker/academic.Dockerfile
    env_file: .private/academic.env
  campus:
    build:
      dockerfile: docker/campus.Dockerfile
    env_file: .private/campus.env
    volumes:
      - campus-media:/app/media
    depends_on:
      academic:
        condition: service_healthy
```

Figure 3.11: Campus and academic services in Docker Compose

Figure 3.11 shows the service definitions used by the project. The campus API owns social interaction, events and application sessions; the private academic API owns AMS integration. Separate environment files hold private configuration and the media volume survives container recreation. Campus startup waits for the academic health check. Login and report work use bounded admission instead of unlimited parallel source requests. The delivered academic topology uses one worker because pending university verification clients are process-local; multi-host replication requires an explicit session strategy.

3.3.10 Deployment of Microservices

```
RECORDED LOCAL DOCKER BUILD
#21 exporting layers
#21 exporting layers 0.9s done
#21 writing image sha256:977
    d81295bd43f98dbbc62976844268956a34a98486a139824149b25f997ba36 done
#21 naming to docker.io/library/veltech-smtp-qa-campus:latest done

LOCAL SMTP CHECK
OK SMTP email configuration is structurally valid. Credentials are hidden.
NOTE Set CAMPUS_PUBLIC_URL to your public HTTPS origin to include app links in emails.
OK SMTP connection and configured security/authentication checked. No email has been
    sent by this verification step.
OK The provider accepted the requested test message. Check the recipient inbox and Spam
    folder to confirm delivery.
```

Figure 3.12: Recorded Docker build and local SMTP verification

Figure 3.12 shows recorded output from the project's Docker build and a local SMTP verification. The resulting campus image was checked against the final mail source and delivered both HTML and plain text to the local test inbox. Windows setup uses Docker Desktop, PowerShell and Atlas without a separate Ubuntu installation. Public operation additionally needs HTTPS, the correct browser origin, persistent keys/media and real provider configuration. This local evidence is not a claim of public university deployment or Internet inbox delivery.

Chapter 4

TESTING

4.1 Testing

The supplied project record reports 607 passing tests: 163 frontend, 253 campus API, 180 Python academic-service and 11 Android configuration/packaging checks, with no failures or skips. It also records production-build and Docker health checks. These are previously recorded results, separate from fictional browser checks and historical live AMS observations.

4.1.1 Test Result

Table 4.1 summarizes representative verification outcomes.

Scenario	**Recorded outcome**
Session continuity	Refresh/session restoration, transient retry and explicit logout verified.
Academic results	Credit joins, published summaries, selected-scale estimates, filters, PDF/Excel and keyboard graph tested.
Polling	Zero hidden requests and zero overview reloads for unchanged sync state in focused checks.
Phone layout	Login, Events and About checked at 320/390/1440 pixels in light/dark themes.
Permissions and notifications	Owner/staff boundaries, room changes, feedback retry and alert deduplication verified.
Work admission	Configured login/report capacities and cancellation checked with synthetic owners.

Table 4.1: Representative verification outcomes

Earlier authorized reads covered 21 profile fields, five courses, 173 result/assessment rows and 71 credit rows for one account. Attendance evidence included 22 timetable entries, five subject summaries and 284 dated marks. No supported fee-ledger or Course Material rows were available. These observations are account-specific.

4.1.2 Test Bugs

Regressions cover academic joins, session restoration, stale responses, polling, mobile layout and duplicate alerts. Live deployment, physical-device evidence, background notifications, provider integrations and failover require separate validation.

Chapter 5

WEBSITE LAUNCH

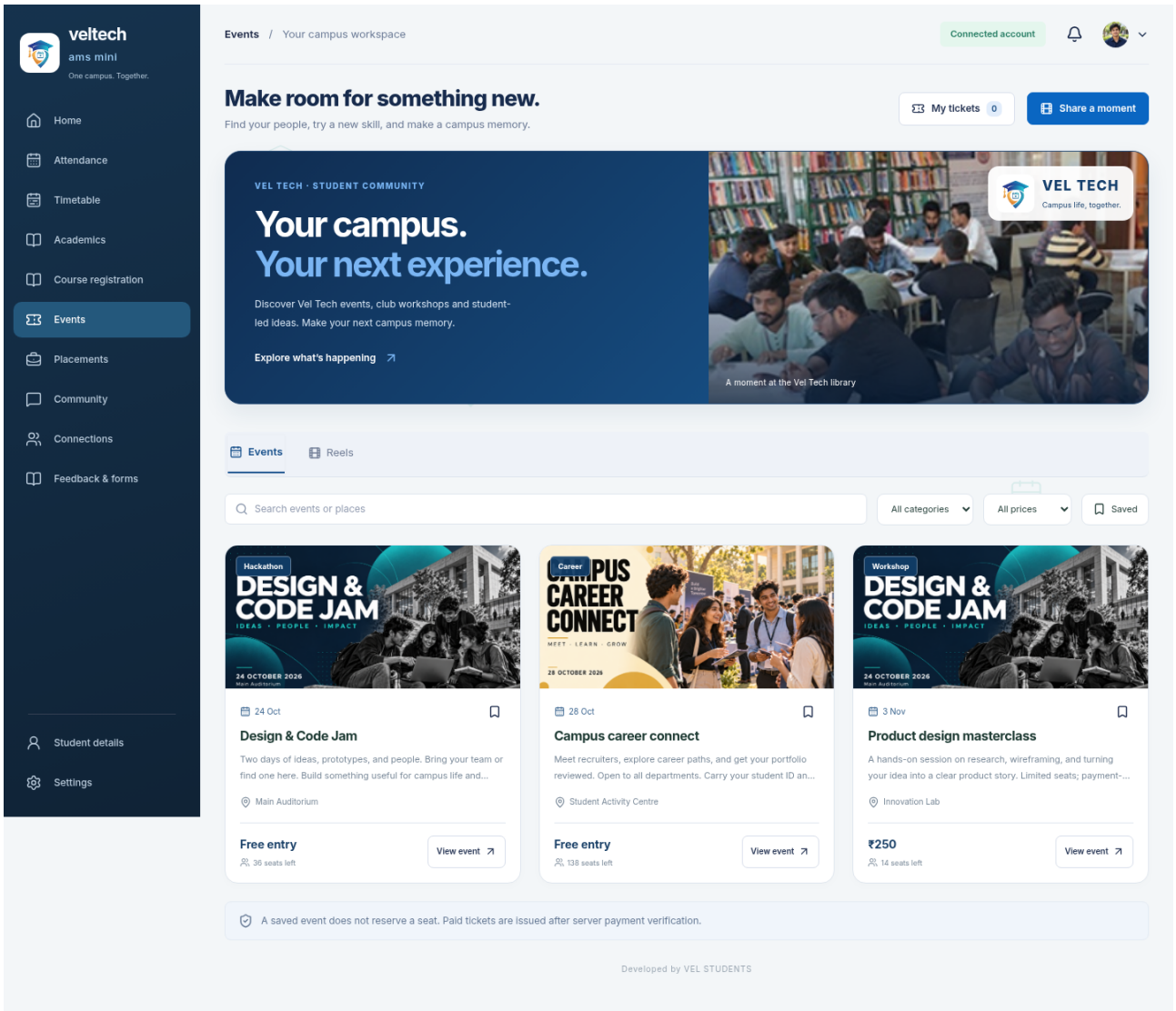


Figure 5.1: Local web application preview

Figure 5.1 shows the local Events preview with fictional records. Windows setup uses Docker Desktop and PowerShell with MongoDB Atlas. The campus API and private academic service must run alongside the interface.

Public operation additionally requires a configured HTTPS origin, persistent keys and media, real provider settings and health checks. The UI preview does not establish a working private AMS session or a public university deployment.

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Website performance

Figure 3.6 shows the cumulative results graph. Filters preserve source semester and report divisions; published values take precedence. Eligible credits c_i and grade points g_i support

$$\mathrm{GPA} = \frac{\sum_i c_i g_i}{\sum_i c_i}, \qquad \sum_i c_i > 0. \tag{6.1}$$

Cumulative totals span eligible semesters. The requested conversion is

$$\mathrm{Percentage} = (\mathrm{CGPA} - 0.5) \times 10. \tag{6.2}$$

Published percentages override this estimate; incomplete or conflicting records prevent a complete calculated result.

Demonstration	**Credits**	**Credit points**	**GPA/CGPA**
Semester I	6	44	7.33
Semester II	4	40	10.00
Cumulative	10	84	8.40

Table 6.1: Fictional credit-weighted example: 8.40 CGPA gives 79%

6.2 Security

Owner/role checks, CSRF protection, HttpOnly sessions and authenticated token encryption protect private work [10, 11]. Official records remain separate from social edits. Private academic responses are excluded from the offline shell cache; Downloads stores only explicit account-scoped copies.

6.3 Responsiveness and mobile-friendliness

Phone navigation, wrapping content and contained scrolling support narrow screens. Subject routes, reels and in-context comments preserve continuity. Device, provider and public-load validation remain deployment tasks.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

Smart College Management Platform consolidates the two existing projects into one academic and campus workspace. Source-aware attendance, timetable, results and credits share navigation with events, social interaction, faculty forms and notifications. Official records remain separate from editable campus content.

The application provides persistent session restoration, incremental synchronization, result calculations and mobile readability. Its recorded automated/browser/Docker checks support the delivered behavior. Missing source data is identified rather than invented; university registration writes and external provider operations retain their documented boundaries.

7.2 Future Enhancements

Priorities include a documented university API agreement, more authorized source-layout fixtures, deployment-specific load/failure testing and backup recovery. Multi-host academic replication needs an explicit university-session strategy and shared media design. Real email, merchant and physical-phone workflows need operator validation.

A signed Android package can follow once HTTPS hosting and signing inputs are configured. Accessibility and moderation controls can expand further. Trained recommendations should be considered only with consented data and measured evaluation; the current community feed uses heuristics [12]. Placement/source modules should grow only as reliable authorized records become available.

Chapter 8

SOURCE CODE

```
import crypto from 'node:crypto';
export const hashToken = token => crypto.createHash('sha256').update(String(token)).digest('hex');
export const newToken = () => crypto.randomBytes(32).toString('hex');
export function checkWebhook(rawBody,signature,secret) {
  if (!Buffer.isBuffer(rawBody) || !secret || typeof signature!=='string' || !/^[a-f0-9]{64}$/i.test(signature)) return false;
  const expected=crypto.createHmac('sha256',secret).update(rawBody).digest();
  return crypto.timingSafeEqual(expected,Buffer.from(signature,'hex'));
}
function keyBuffer(key) {
  if (!/^[a-f0-9]{64}$/i.test(key || '')) throw new Error('CAMPUS_LINK_KEY must be a persistent 32-byte hex key');
  return Buffer.from(key,'hex');
}
export function encryptToken(token,key) {
  const iv=crypto.randomBytes(12),cipher=crypto.createCipheriv('aes-256-gcm',keyBuffer(key),iv);
  const body=Buffer.concat([cipher.update(token,'utf8'),cipher.final()]);
  return Buffer.concat([iv,cipher.getAuthTag(),body]).toString('base64');
}
export function decryptToken(sealed,key) {
  const bytes=Buffer.from(sealed,'base64');
  const cipher=crypto.createDecipheriv('aes-256-gcm',keyBuffer(key),bytes.subarray(0,12));
  cipher.setAuthTag(bytes.subarray(12,28));
  return Buffer.concat([cipher.update(bytes.subarray(28)),cipher.final()]).toString('utf8');
}
```

Figure 8.1: Selected campus-service security source

Figure 8.1 reproduces selected project helpers for hashing session tokens, checking configured webhook signatures and encrypting private academic link tokens. Keys come from persistent server configuration; no secret value is shown. The full application keeps React under `frontend/`, campus routes under `services/campus/` and academic integration under `services/academic/`. The excerpt illustrates selected helpers rather than the complete service.

Chapter 9

SCREENSHOTS

These additional project views show campus events and student community. All visible student and event values are fictional demonstration records.

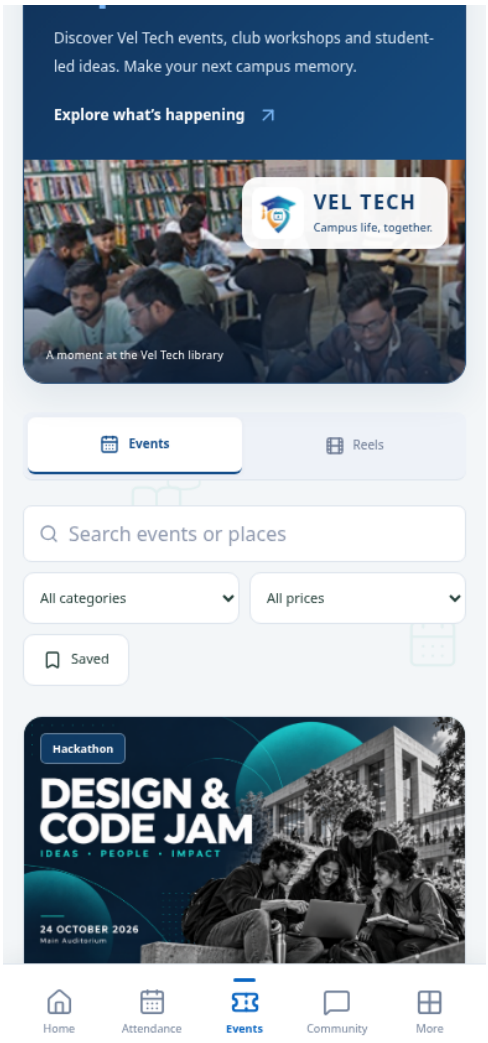


Figure 9.1: Mobile event discovery

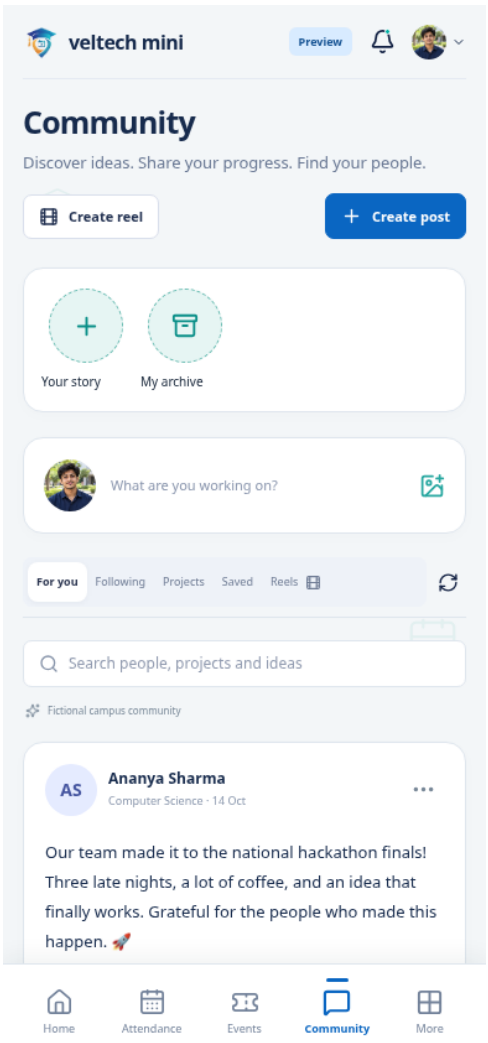


Figure 9.2: Mobile community feed

Figures 9.1 and 9.2 show the shared phone navigation, event discovery, posts, reels and profile context. Registration opens the selected event's workflow, while comments and social actions remain associated with the active content. Academic, login and registration screens are illustrated individually in Chapter 3.

References

[1] Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, "Official university website." Accessed: Oct. 21, 2026. [Online]. Available: https://www.veltech.edu.in/

[2] Vel Tech, "Academic Management System." Accessed: Oct. 21, 2026. [Online]. Available: https://ams.veltech.edu.in/Default.aspx

[3] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601–618, 2010, doi: 10.1109/TSMCC.2010.2053532.

[4] MongoDB, "Atlas free-cluster limitations." Accessed: Oct. 21, 2026. [Online]. Available: https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/

[5] React, "React documentation." Accessed: Oct. 21, 2026. [Online]. Available: https://react.dev/

[6] Vite, "Vite guide." Accessed: Oct. 21, 2026. [Online]. Available: https://vite.dev/guide/

[7] FastAPI, "FastAPI documentation." Accessed: Oct. 21, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[8] Docker, "Docker Compose." Accessed: Oct. 21, 2026. [Online]. Available: https://docs.docker.com/compose/

[9] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *Proc. IEEE 9th Int. Conf. Service-Oriented Computing and Applications (SOCA)*, 2016, pp. 44–51, doi: 10.1109/SOCA.2016.15.

[10] OWASP Foundation, "Session management cheat sheet." Accessed: Oct. 21, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[11] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996, doi: 10.1109/2.485845.

[12] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, 2005, doi: 10.1109/TKDE.2005.99.